

First, connect the VCC, GND, TX, and RX pins of the CH9102 USB-to-UART module to the corresponding pins on the assembled PCB. Then, connect the CH9102 USB-to-UART module to the laptop via USB to power the PCB and establish serial communication.

PARTS LIST (FOR ONE SET)

Semiconductors:

LD1	- Laser diode
T1	- BC547 NPN transistor
D1	- BPW34 pin photodiode
IC1 (IC1A-IC1C)	- LM393 comparator

Resistors (all 1/4-watt, $\pm 5\%$ carbon):

R1	- 2k Ω
R2	- 1k Ω
VR1	- 2k Ω potentiometer

Capacitors:

C1	- 0.1 μ F ceramic disc
----	----------------------------

Miscellaneous:

U1	- CH9102 USB to TTL converter module
	- PCB (designed or custom-made) or Breadboard
	- Connecting/jumper wires
	- Enclosure (optional)

After optimising the hardware and software, the communication speed was set to 520kbps, which proved to be the highest stable data rate.

At this speed, the bit period is approximately 1.9 μ s, very close to the practical operating limit of the LM393 comparator (typical response time of about 1.3 μ s under suitable conditions). During laboratory testing, baud rates of up to 750kbps were achieved without errors under ideal conditions. However, this performance may not remain reliable under different ambient conditions or with slight optical misalignment. Therefore, 520kbps is recommended for dependable operation.

Reliable high-speed communication requires proper adjustment of the VR1 threshold on both modules, accurate optical alignment, adequate filtering, and minimal electrical noise and crosstalk. Data integrity

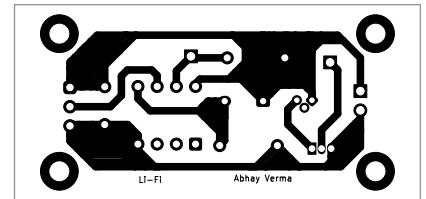


Fig. 5: Actual-sized PCB of the Li-Fi transceiver module

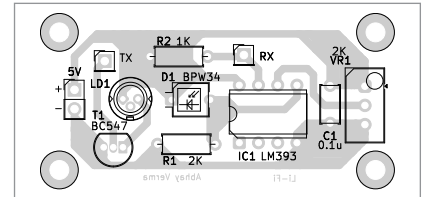


Fig. 6: Components layout of the PCB

TO DOWNLOAD SOURCE CODE AND RESOURCES



Scan this QR code or go to <https://sourcecode.electronicsforu.com> to access and download source code and other resources for this project

```
import serial
import time
import os

PORT = "COM31"
BAUD = 520000
PACKET_SIZE = 128

ser = serial.Serial(PORT, BAUD, timeout=1.0)
time.sleep(1)
ser.reset_input_buffer()
ser.reset_output_buffer()

print("Waiting for start...")

line = ser.readline()
while b"<START>" not in line:
    line = ser.readline()

print("Start detected")

filesize = int(ser.readline().strip())
packet_size = int(ser.readline().strip())
print("File size:", filesize)
print("Packet size:", packet_size)

received = 0
expected_packet_no = 0

def checksum(packet):
    return sum(packet) % 256

with open(r"C:\Users\Abhay Verma\Desktop\received1.zip", "wb") as file:
    while received < filesize: #synchronization check
        sync1 = ser.read(1)
        if sync1 != b'\xAA':
            continue

        sync2 = ser.read(1)
        if sync2 != b'\x55':
            continue

        packet_no_bytes = ser.read(1) # Header found, read metadata
        length_bytes = ser.read(1)

        if not packet_no_bytes or not length_bytes:
            continue # Data cut off, transmitter will timeout and resend

        packet_no = packet_no_bytes[0]
```

Fig. 4: Snippet of the receiver Python code

is ensured through packet framing, checksum-based error detection, and automatic packet retransmission. Any lost or corrupted packets are resent until the complete file is received correctly, enabling reliable bi-directional wireless file transfer over the optical Li-Fi link.

Finally, the system demonstrates how digital data can be transmitted using light through on-off keying and received by a photodiode. It also demonstrates high-speed UART communication, data packetisation with checksum-based error checking and automatic retransmission, as well as the importance of comparators, filtering, and component selection for reliable signal reception. It also shows how discrete electronic circuits and software protocols work together to create a complete bi-directional Li-Fi communication system. **EFY**

Abhay Verma is an electronics engineer at EFY and a passionate electronics hobbyist with a keen interest in practical circuit design and DIY projects.